# SMP construction

> **Status 2026-10-06.** This document explains how the SMP construction is built; its technical content still applies, its repository layout predates the devices dissolution. The migration from `xpack-development` to `smp` was then executed with the commit-only procedure of `xpack-dev-smp.md` Part II: no `scripts/smp/`, no chunk recipes, one subject per commit, only added or modified files. The result is recorded in its Part III: 343 commits over `micro-os-plus-iii`, `-posix-arch`, `-cortexm`, `-aarch32` and `-aarch64`; `micro-os-plus-iii-devices` dissolved into the ports' `drivers/` and `soc/<chip>/` (no `UOS_DEVICES_DIR`); every commit built; the full xpm test run (native, QEMU Cortex-M, `test-smp-cmake`) green. Where this document disagrees with it (step or PR numbering, scripts, the devices repository, test counts), `xpack-dev-smp.md` applies.

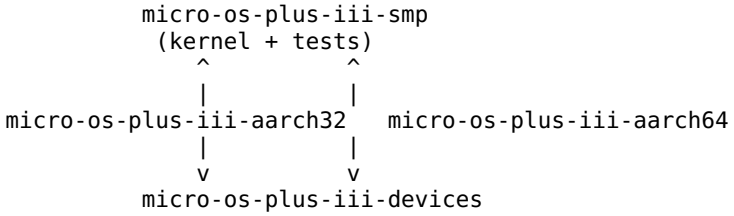
How the µOS++ III SMP kernel, an architecture port, the device drivers and the test applications fit together, and what each one owes the others.

---

## 1. The six repositories

```
micro-os-plus-iii-smp          the kernel, the shared tests, the shared build
rules
micro-os-plus-iii-devices      SD, flatfs, DWC2, FatFs, BCM2837 SoC support
micro-os-plus-iii-aarch32      ARMv7-A port
micro-os-plus-iii-aarch64      ARMv8-A port
micro-os-plus-iii-cortexm      ARMv7E-M / ARMv8-M port
micro-os-plus-iii-posix-arch   native host port
```

Dependencies run **one way**. An architecture project consumes the kernel and, if it needs them, the devices; neither of those knows an architecture project exists. Nothing is vendored: the projects are cloned side by side and CMake finds them as siblings.




Keeping the kernel tree free of device and board code is what lets `src/` and `include/` match upstream micro-os-plus-iii path for path, so the fork can be merged against upstream later with conflicts only in the files that genuinely differ.

## 2. Division of labour

| Concern | Lives in | Why there |
|---|---|---|
| Scheduler, threads, sync objects, per-CPU run state | kernel `src/rtos/` | no machine instructions; identical for every port |
| Type widths, IRQ state representation, core count | port `include/cmsis-plus/rtos/port/os-c-decls.h` | the only place a word size is decided |

| Concern | Lives in | Why there |
|---|---|---|
| Instructions: barriers, IRQ mask, core id, exclusive monitor | port `include/cmsis-plus/rtos/port/os-inlines.h` | one file per ISA |
| Bring-up, kernel lock, context switch hooks | port `src/rtos/os-core.cpp` | the port's half of the scheduler |
| Reset vector, MMU, exception vectors, timer | port `src/` | board and ISA |
| SD, FatFs, flatfs, USB, SoC mailbox | devices repo | shared verbatim by both ARM ports |
| Test applications | port `test/<board>/` | every board owns its own — see `tests-in-aarch32-aarch64.md` |

### Two files called `src/rtos/os-core.cpp`

This is deliberate and both are compiled into every image.

- The **kernel's** is the portable SMP scheduler: per-CPU `current_thread_[]`, the affinity picker, the per-core idle bookkeeping. It contains no machine instructions and calls out to `port_cpu_id()`.
- The **port's** is the bring-up and the lock: secondary core release, the recursive kernel spinlock, the context-switch entry points.

They share a path because each is at the path its own project expects; they do not collide, and the predecessor Makefiles compiled both in exactly the same way.

## 3. The SMP port contract

The whole kernel-side SMP patch is 379 lines and asks a port for a small, closed set of things.

### 3.1 Types — `cmsis-plus/rtos/port/os-c-decls.h`

```
typedef uint64_t os_port_clock_timestamp_t;
typedef uint32_t os_port_clock_duration_t;
typedef uint64_t os_port_clock_offset_t;
typedef bool     os_port_scheduler_state_t;
typedef uint64_t os_port_irq_state_t;                  /* AArch32: uint32_t */
typedef uint64_t os_port_thread_stack_element_t;       /* AArch32: uint32_t */
typedef uint64_t os_port_thread_stack_allocation_element_t;

typedef struct { os_port_thread_stack_element_t* stack_ptr; }
        os_port_thread_context_t;

typedef struct { volatile uint32_t lock, owner, depth; } /* kernel lock  */
typedef struct { volatile uint32_t lock; }               /* ticket lock  */
```

plus the configuration macros in section 5.1.

### 3.2 Declarations — `cmsis-plus/rtos/port/os-decls.h`

The C++ half: the `port::stack`, `port::interrupts` and `port::scheduler` namespaces, and the four objects a port must **define**:

```
extern volatile state_t  lock_state[OS_NCPU];
extern volatile state_t  kernel_lock_state[OS_NCPU];
extern smp_klock_t       _smp_klock;
```

```
extern smp_tlock_t       _smp_tlock;
extern volatile unsigned _port_ctx_pending[OS_NCPU];
```

`kernel_lock_state[]` (2026-10-23) is declared for every port but defined and used only by a port that also defines `OS_HAS_PORT_KERNEL_LOCK` (section 3.3).

This file contains no machine instructions, and the two AArch ports had it byte-identical but for one missing `volatile` (cortexm keeps its own, larger `os-decls.h`). It therefore lives **once**, in the kernel repository at `port/smp-common/`, and a port opts in:

```
target_link_libraries (my-port INTERFACE micro-os-plus::port-smp-decls)
```

It is on no default include path. A port with different needs writes its own `os-decls.h` in its own `include/` and simply does not link that target, so the two can never be confused.

The kernel's `include/cmsis-plus/rtos/os-decls.h` also declares the port's `port::clock_highres` contract (`has_hardware_counter()` and `hardware_counter()`), while `include/cmsis-plus/rtos/os-thread.h` defines `thread::state::destroying = 7` to arbitrate SMP thread teardown between `thread::kill()` and `os_rtos_idle_actions()`.

## 3.3 Inline operations — `cmsis-plus/rtos/port/os-inlines.h`

Per ISA. The kernel calls, and the port supplies:

| Function | AArch32 | AArch64 |
|---|---|---|
| `port_cpu_id_inline()` | `mrc p15, 0, Rd, c0, c0, 5` | `mrs Xd, mpidr_el1` |
| `interrupts::critical_section::enter/exit` | `cpsid i` / `cpsie i` | `msr daifset/daifclr, #2` |
| `interrupts::uncritical_section::enter/exit` | as above, inverted | as above, inverted |
| `scheduler::lock/unlock/locked` | recursive kernel lock | per-core `lock_state[cpu]` only (the classic scheduler lock) |
| `scheduler::kernel_lock/kernel_locked` | — (the kernel falls back to the scheduler lock) | `interrupts::critical_section` + per-core `kernel_lock_state[cpu]` |
| `_smp_klock_raw_acquire/release` | `ldrex/strex` | `ldaxr/stlxr` |
| `clock_highres::has_hardware_counter()` | `true` | `true` |
| `clock_highres::hardware_counter()` | `timer_arm::get_count()` (CNTPCT) | `timer_arm::get_count()` (CNTPCT_EL0) |

**The kernel's own critical section (2026-10-23).** The kernel protects its data with `scheduler::kernel_critical_section` (and `kernel_uncritical_section`, `kernel_lockable`). A port that defines `OS_HAS_PORT_KERNEL_LOCK` in `os-c-decls.h` supplies `port::scheduler::kernel_lock()` and `kernel_locked(state)`; its `locked()` query must return true under either lock. The aarch64 port does, which leaves its `scheduler::lock()` the classic per-core lock (IRQs on, no spinlock). Without the macro the kernel uses the scheduler lock, as AArch32 does. Either way, a `reschedule()` asked for under a lock must not be lost: the aarch64 port keeps it in `resched_pending[cpu]` and switches when the last lock is released.

## 3.4 Functions with C linkage

```
unsigned port_cpu_id (void);          /* current core index, 0..OS_NCPU-1   */
void     port_sys_init (void);        /* per-core tick + IRQ controller     */
void     port_ctx_switch (void);      /* context-switch entry              */
void     port_smp_ipi (unsigned cpu); /* send reschedule IPI to target CPU  */
```

```
void     port_smp_secondary_start (void);  /* secondary core entry point   */
void     smp_install_boot_threads (void);  /* supplied by the application  */
```

`smp_install_boot_threads()` is the one the *application* owes the port: it creates an idle thread per secondary CPU and registers each in `os::rtos::scheduler::os_idle_thread_core[]` before the cores are released. Most of a board's tests get it from that board's `test/<board>/src/test-smp-boot.cpp`; a test carried over whole from the predecessor brings its own, and says so through `BOARD_TEST_SELF_CONTAINED` in the board's `test/<board>/tests.cmake`.

### 3.5 Bring-up order

```
reset  ->  startup.S            core 0 only; others park
       ->  os_main              application
       ->  interrupts::uncritical_section::enter ()   unmask IRQs
       ->  smp_install_boot_threads ()                idle thread per core
       ->  smp::start_secondary_cores ()              release cores 1..N-1
       ->  scheduler::start ()
```

Each secondary enters `port_smp_secondary_start()`, takes the kernel lock, initialises its tick through `port_sys_init()`, and schedules its idle thread.

## 4. What makes the test applications ISA-neutral

One copy of each test is compiled by every port that can run it. The disagreements between the old 32b and 64b copies resolved like this:

| Was | Now | Note |
|---|---|---|
| `dsb` / `dmb` vs `dsb sy` / `dmb ish` | `dsb sy`, `dmb ish` | not a difference: the explicit forms assemble on ARMv7-A as well |
| `cpsie i` vs `msr daifclr, #2` | `interrupts::uncritical_section::enter()` | a kernel API every port implements |
| `mrrc p15,…,c14` vs `mrs cntpct_el0` | `timer_arm::get_count()` | the port already had it |
| `"(AArch64)"` in a banner | `PORT_BANNER_ISA` | supplied by the port's `uart.hpp` |

The rule: **an application names no register and no instruction that is not portable across the ports that can run it.** What it needs from a port it takes through the port's headers.

## 5. Preprocessor defines, per folder

### 5.1 `micro-os-plus-iii-aarch32` / `-aarch64` — defined by the port header

Set in `include/cmsis-plus/rtos/port/os-c-decls.h`; the build may override the first two.

| Macro | Value | Meaning |
|---|---|---|
| `OS_NCPU` | 4 (`#ifndef`) | core count; the build chooses |
| `OS_USE_SMP_SCHEDULER` | 1 when `OS_NCPU > 1` | selects the SMP scheduler over the scalar one |
| `SMP_NO_OWNER` | `0xFFFFFFFFu` | "kernel lock unowned" sentinel |
| `OS_SMP_IPI_SGI` | 0 | |

| Macro | Value | Meaning |
|---|---|---|
| | | cross-core reschedule signal; BCM2837 has no GIC, so local mailbox 0 |
| `OS_INTEGER_RTOS_STACK_FILL_MAGIC` | `0xEFBEADDE` / … `EFBEADDEULL` | stack paint; element width |
| `OS_HAS_INTERRUPTS_STACK` | defined | the port provides a separate IRQ stack |

## 5.2 Set by the build for every application

`uos_add_app()` in `cmake/uos-app.cmake`:

| Macro | Source |
|---|---|
| `OS_NCPU=<n>` | `NCPU` argument |
| `OS_USE_SMP_SCHEDULER=1` | added automatically when `NCPU > 1` |

The architecture project's `test/CMakeLists.txt`:

| Macro | Where | Meaning |
|---|---|---|
| `TRACE` | both | enable the trace channel |
| `__ARM_EABI__` | both | ARM EABI |
| `__ARM_ARCH_7A__` | aarch32 only | ARMv7-A |
| `SEMIHOST` | both | mirror the console to the semihosting channel and exit through it once a RESULT is printed. **On by default**, as it was in the Makefiles, so no build is silently non-semihosting |
| `BOARD_RPI3B` | `-DBOARD=rpi3b` | Pi 3 B instead of Pi Zero 2 W: other linker script, other banner |
| `HW_BUILD` | `hwd` variant | SD tests use the existing FAT32 boot partition through FatFs instead of formatting a blank card |
| `DEBUG_BOOT` | `-DUOS_DEBUG_BOOT=ON` | early-boot asm markers from `startup.S`, for bring-up under OpenOCD. **Off by default**, as the sources assume: each marker is a semihosting trap, and under a JTAG probe a trap costs real time |
| `LED_PIN` | `BOARD=rpi-zero-2w` | the GPIO driven as the user LED, 29 by default — the Zero 2 W's onboard green ACT LED. `led.hpp`'s own fallback is GPIO 16, header pin 36, which blinks nothing on a bare board. Which pin it is depends on the **board**, so it is set here for every test rather than per application; `BOARD=rpi3b` does not use it at all, because there the ACT LED is VideoCore expander GPIO 130 behind the mailbox |

## 5.3 Per-application, from the board's `test/<board>/tests.cmake`

On the Pi boards two applications have knobs. `cmsis-os-validator` gets `UOS_CMSIS_OS_VALIDATOR`, which makes the board's sources map a "no cycle counter" page where the validator probes the Cortex-M DWT and route the local Mailbox 1 interrupt to the validator's handler (its stand-in for NVIC IRQ 0); no other image changes. `usb_test` has these:

| Macro | Default | Meaning |
|---|---|---|
| `LED_BLINKS` | 3 | blinks per burst |
| `LED_ON_MS` / `LED_OFF_MS` | 40 / 40 | half-cycles; the tick is 1 ms, so each is clamped to at least that |
| `LED_GAP_MS` | 300 | dark gap between bursts |
| `USB_FORCE_FS` | defined | force full speed. It also sets the driver's `kMpsHs` to 64, which is what bounds a bulk transfer: `D{I,0}EPTSIZ.PKTCNT` is 10 bits on this core, so 1023 packets — 65472 bytes at full speed against 523776 at high speed |

Consumed but not set by default:

| Macro | Meaning |
|---|---|
| `RUN_MS` | wall-clock run length for `smp-num-test`, `smp-pipeline-test`, `smp-pro-cons-test`. The sources carry their own defaults; set it for short QEMU smoke runs |
| `TEST_IDLE_STACK_WORDS` | stack words per secondary idle thread, default 512 |

## 5.4 `micro-os-plus-iii-smp` — the kernel

The kernel reads a large set of optional `OS_*` switches (`OS_TRACE_RTOS_*`, `OS_INCLUDE_RTOS_STATISTICS_*`, `OS_USE_RTOS_PORT_*`, …). **None is required.** The ARM builds set none of them; every one defaults to off.

Two are structural:

| Macro | Meaning |
|---|---|
| `OS_USE_OS_APP_CONFIG_H` | every kernel TU includes `<cmsis-plus/os-app-config.h>` from the port. The ARM ports keep that file empty and do not define this |
| `OS_USE_SMP_SCHEDULER` | selects the SMP scheduler. Comes from the port header or the build, never hand-written in a source |

## 5.5 `micro-os-plus-iii-devices`

FatFs is configured entirely through `fatfs/ffconf.h` (`FF_*`, ~40 macros); nothing needs to be passed on the command line. The only build-visible macro is `USB_FORCE_FS`, which `usb_test` sets.

Device sources dispatch on `__aarch64__` inline where an instruction genuinely differs — three places in total, for the cache maintenance and the core-id read. No project-specific macro selects an ISA.

# 6. Verifying a port

```
tools/verify-kernel-compiles.sh [--without <group>]... \
    <port-include-dir>[:<dir>...] [compiler [flags...]]
```

Several directories, :-separated, are searched in that order: a port whose board overlays one on another (cortexm's `include-rp2350:include`), or the AArch ports with the kernel's `port/smp-common`. A bare-metal port passes its cross compiler, followed by the facts a board would supply (aarch32: `-mcpu`, `OS_NCPU`, `OS_SMP_IPI_SGI`, `PORT_GREETING`). `--without startup` skips the optional `startup` group, which the AArch ports do not link; the skipped sources are reported, never counted. `.c` sources compile as C.

Compiles every source the kernel declares against that port's headers. It is the only meaningful standalone check: the kernel can never compile on its own, because `include/cmsis-plus/rtos/os-decls.h` includes `<cmsis-plus/rtos/port/os-decls.h>`, which only a port supplies.

Current result on `posix-arch` and `aarch64`: **61 declared, 61 compiled**; on `aarch32`, 57 compiled and the 4 `startup` sources skipped. The exact commands are in `STATUS.md`.